Word Embeddings — Theory and Analysis

PRASANNA KOIRALA · AIAC 536 NATURAL LANGUAGE PROCESSING · KATHMANDU UNIVERSITY

Introduction

A computer cannot understand the word *cat*. It can only do arithmetic. So every word must first be turned into numbers.

A **word embedding** is exactly that — a word represented as a list of numbers, called a **vector**:

```
"cat"  →  [ 0.21, −0.44, 0.87, ... ]
```

What matters is *which* numbers. Chosen well, **words with similar meanings get similar numbers**, so a computer can tell that *cat* is closer to *dog* than to *rocket* — without being told what any of them mean.

1 One-Hot Encoding vs Dense Embeddings

One-hot encoding. Each word becomes a vector as long as the whole vocabulary: all zeros, with a single 1 marking its position.

```
Vocabulary = [ cat, dog, king, queen, rocket ]

cat   = [ 1, 0, 0, 0, 0 ]
dog   = [ 0, 1, 0, 0, 0 ]
king  = [ 0, 0, 1, 0, 0 ]
```

This has two serious problems.

(a) Size. A real vocabulary holds about 50,000 words, so every word becomes 50,000 numbers of which 49,999 are zero. Nearly all storage is wasted.

(b) No meaning. Every pair of different words is equally far apart:

```
cos(cat, dog) = 0        cos(cat, rocket) = 0
```

The encoding claims *cat* is as unrelated to *dog* as it is to *rocket*.

Dense embeddings fix both problems. Each word becomes a short vector of real numbers **learned from text**. Because *cat* and *dog* appear in similar sentences, training pulls their vectors together.

	ONE-HOT	DENSE EMBEDDING
Length	vocabulary size (~50,000)	small and fixed (50–300)
Values	zeros and one 1	real numbers
Created by	indexing	learned from data
Captures meaning	No	**Yes**
Similar words close	no — all equidistant	**yes**

2 Word2Vec, GloVe and FastText

All three rest on the **distributional hypothesis**:

> *"You shall know a word by the company it keeps."*

> — J. R. Firth, 1957

Words used in similar contexts have similar meanings.

Word2Vec (2013)

Slides a window over the text and trains a small neural network. For the sentence `the cat sat on the mat` with centre word **sat**, the context is [the, cat, on, the]. There are two ways round:

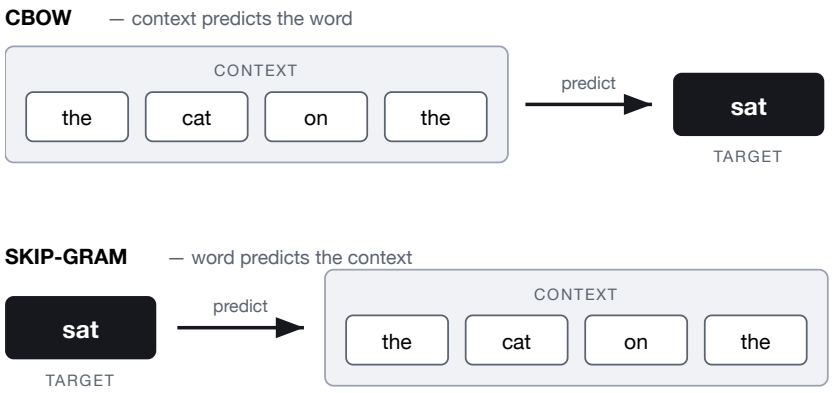

Figure 1. The two Word2Vec architectures. Same information, opposite direction.

	CBOW	SKIP-GRAM
Direction	context → word	word → context
Speed	faster	slower
Rare words	weaker	**better**
Small corpus	weaker	**better**

GloVe (2014) — Global Vectors

Word2Vec sees only one window at a time. GloVe instead first counts how often every pair of words co-occurs across the **whole corpus**, then factorises that count matrix.

> **KEY CONTRAST**
> Word2Vec uses **local** windows. GloVe uses **global** counts.

FastText (2016)

Treats a word as a bag of **character n-grams**:

```
"playing"  →  pla   lay   ayi   yin   ing
"played"   →  pla   lay   aye   yed       ← shares  pla, lay
```

Two advantages follow. An **unseen word** still gets a vector built from its pieces. And **morphology** comes free — *play / played / playing* share n-grams, so they automatically receive related vectors. This matters greatly for morphologically rich languages such as **Nepali**.

MODEL	CORE IDEA	STRENGTH
Word2Vec	predict within a local window	fast, simple
GloVe	factorise global co-occurrence counts	whole-corpus statistics
FastText	word = sum of character n-grams	unseen words, morphology

3 Semantic Similarity

Closeness of meaning is measured by **cosine similarity** — the *angle* between two vectors, ignoring their length:

$$\cos(A, B) = \frac{A \cdot B}{|A| \times |B|}$$

−1	opposite
0	unrelated
+1	identical

Why the angle, and not the distance? Frequent words develop longer vectors. Plain straight-line distance would call a common word "far" from a rare one purely because of frequency. The angle discards length and keeps only direction — and meaning lives in the direction.

Measured on GloVe (100 dimensions, 400,000 words)

PAIR	COS	
cat — dog	**0.88**	same contexts
good — bad	**0.77**	*see warning below*
king — queen	**0.75**	both royalty
cat — rocket	**0.19**	unrelated

> **WARNING — AN IMPORTANT LIMITATION**
>
> *good* — *bad* scores **0.77**, nearly as high as *king* — *queen*, even though they are opposites. Cosine similarity really measures **"appears in similar contexts"**, not "means the same thing". Both words fit the sentence "this film was very ___". So **static embeddings cannot separate antonyms from synonyms.**

Vector arithmetic

Relationships between words appear as consistent **directions** in the space.

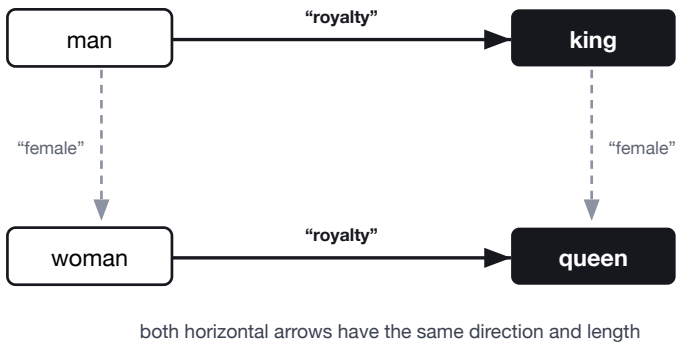

Figure 2. Relationships as directions: `king − man + woman ≈ queen`

ARITHMETIC	TOP RESULT	SCORE
`king − man + woman`	**queen**	**0.77**
`paris − france + nepal`	**kathmandu**	**0.81**

The vectors were never told what a capital city is. They learned the relationship *country → capital* from usage alone, and it transfers correctly to Nepal.

4 The Role of Dimensionality

d is simply how many numbers represent each word.

Too small (d = 5): there is not enough room, so different meanings collide — the model **underfits**. **Too large** (d = 1000): each dimension sees too little data, so the model memorises noise instead of meaning — it **overfits** — and costs more memory and time.

Figure 3. Quality against dimensionality. Both extremes are worse than the middle.

D	EFFECT
`< 20`	underfits — different meanings collide
`50 – 300`	**the usual sweet spot**
`> 1000`	overfits, slower, diminishing returns

> **IMPORTANT**
>
> Individual dimensions are **not interpretable**. Dimension 42 does not mean “royalty”. Meaning is spread across the whole vector and appears as *directions* through the space, not along single axes. This is why embeddings work well but are hard to explain.